Semi-adaptive Synergetic Two-way Pseudoinverse Learning System

A Non-Gradient Deep Learning Approach

Sai Reddy (CB.SC.U4AIE24205) Devana (CB.SC.U4AIE24213)
Manas (CB.SC.U4AIE24257) Zahwa (CB.SC.U4AIE24261)

School of Artificial Intelligence

Semi-adaptive – The network grows its own depth automatically, only when needed.

In our project: We start with 1 layer. After each new layer is added, we check if accuracy improved on the validation set. If it did – **add another.** If it stopped improving – **we stop there.** This is called **Early Stopping** – how our model decides its own depth without anyone manually choosing.

Synergetic – Multiple mini-models work as a team.

In our project: We run multiple subsystems in **parallel**, each trained on a **random sample** of the data, and combine their outputs.

Two-way – Learns FORWARD (data $\rightarrow$ pattern) and BACKWARD (answer $\rightarrow$ feature) simultaneously.

In our project: Each subsystem has a **forward PILAE stack** and a **backward PILAE stack** whose features are **fused together.**

Problem Statement

How Traditional AI Trains – Gradient Descent

Start with **random weights** → check how wrong the answer is → **adjust the weights a little** in the direction that reduces the mistake → repeat thousands of times until the answer is good enough.

Limitations of Gradient Descent

High Computational Cost – requires iterative training, repeating the same data thousands of times.

Sensitive to Hyperparameters – small changes in learning rate or other settings can completely break the training.

Vanishing / Exploding Gradients – in deep networks the error correction information either fades away or grows out of control before reaching the early layers.

Architectural Challenges

Manual Depth Selection – choosing the optimal number of layers is done by hand with no automatic guidance.

Fixed Architecture – a network that is too small underfits, too large overfits. No way to adapt automatically.

Why Pseudoinverse Learning?

Problem: Slow iterative training

Pseudoinverse calculates the best weights in **one single step** using the Moore-Penrose formula directly – no repeated passes, no thousands of iterations.

$$W_d = XH^\top (HH^\top + \lambda I)^{-1}$$

Problem: Sensitive to hyperparameters

There is **no learning rate**, momentum or batch size. The output weights are determined analytically. Only λ controls regularisation and it is easy to set.

Problem: Vanishing / Exploding Gradients

There is **no gradient at all**. Weights are calculated directly layer by layer using the pseudoinverse. The error correction information never travels backwards through layers so this problem simply does not exist in our system.

Why Pseudoinverse Learning?

Problem: Manual architecture / Fixed depth

We use **Early Stopping**. The model starts with 1 layer and adds more only when accuracy improves on the validation set. Depth is decided **automatically** by the data, not by hand.

What we use in our project

Each subsystem is built using **stacked PILAE** – multiple pseudoinverse autoencoders trained layer by layer. The output of each layer becomes the input of the next, building richer and more abstract features at every level.

Methodology Overview

The Big Picture

The system is built as a **Synergetic Learning System (SLS)** – multiple elementary models working together, each one being a hybrid neural network that processes data at different levels.

Each Elementary Model has 3 modules

1. Forward Learning – data flows in and is compressed layer by layer to extract patterns. Built using stacked PILAE blocks trained with the pseudoinverse formula.

2. Backward Learning – starts from the correct label and works backwards through the same architecture to extract label-aware features that forward learning misses.

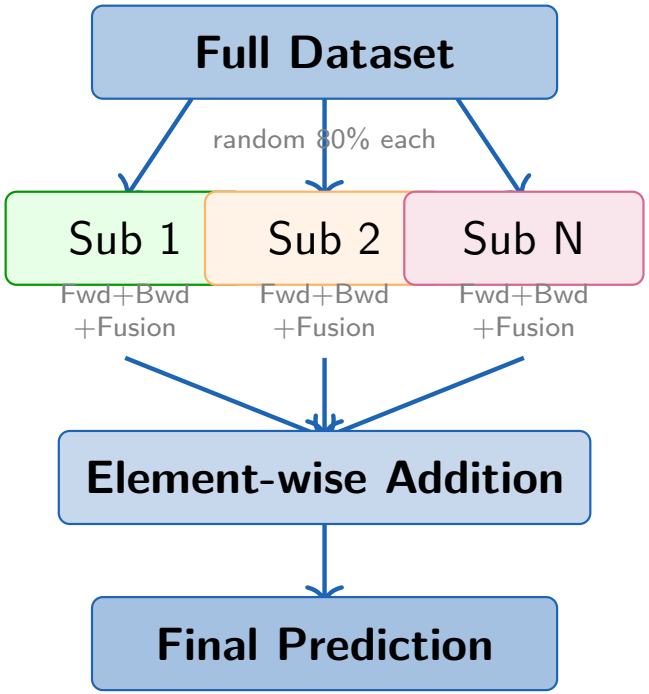
3. Feature Fusion – the features from forward and backward learning are concatenated together into one rich representation for the final classifier.

How the Subsystems Work Together

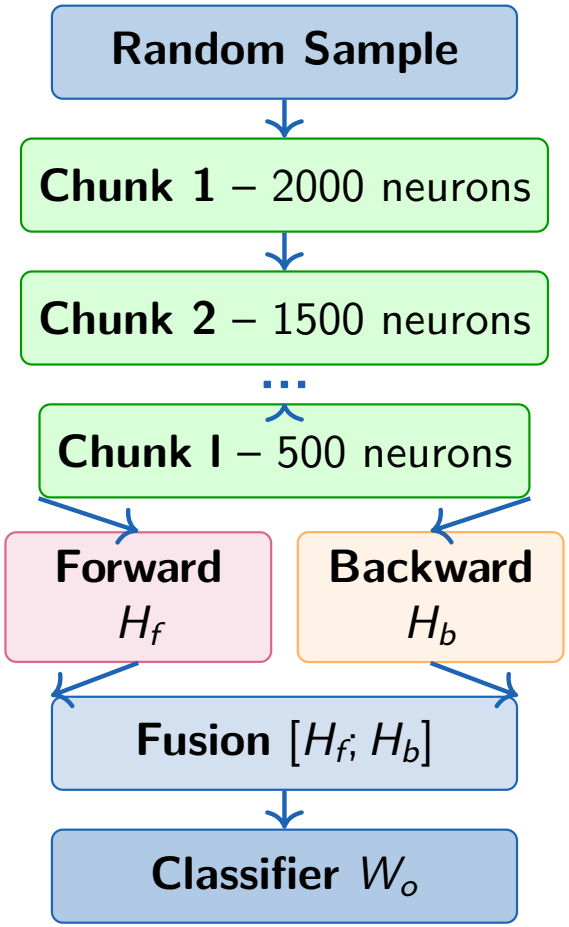
Each subsystem trains on a **random sample** of the data independently and in parallel. Their outputs are combined using **element-wise addition** into one final prediction.

Building the System – The Full Picture

How Data Splits Into Subsystems

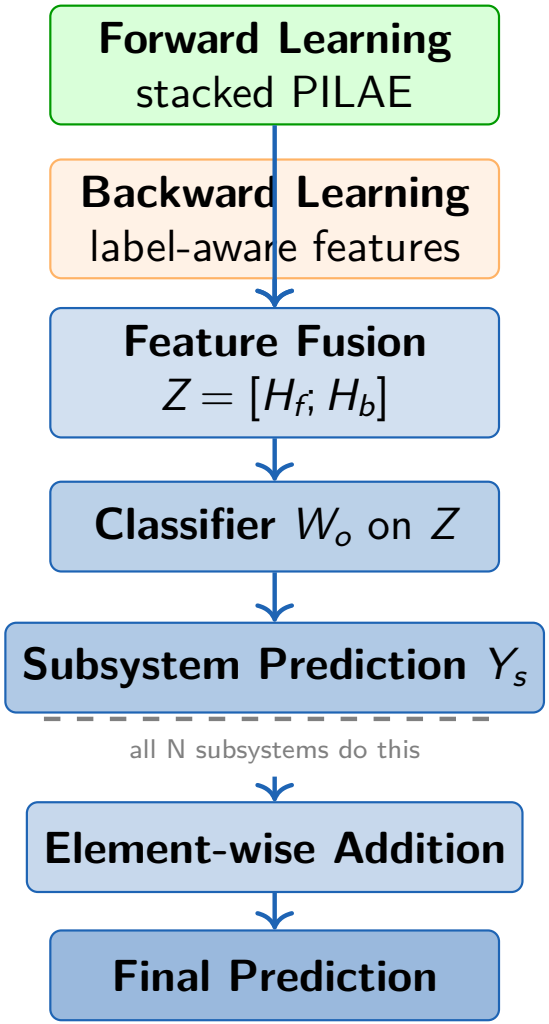


Inside ONE Subsystem



What Happens After Feature Fusion?

Inside ONE Subsystem – Full Flow



Step 1 – Feature Fusion

Forward features H_f and backward features H_b are **concatenated** into $Z = [H_f; H_b]$. Forward captures data patterns. Backward captures class-specific features.

Step 2 – Classifier

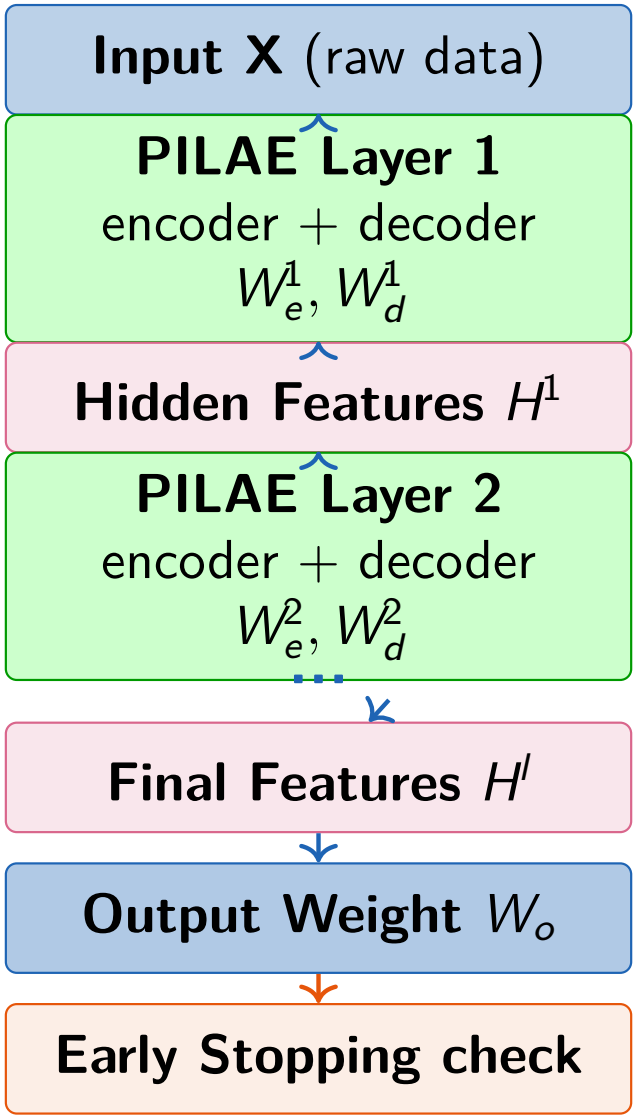
W_o computed **analytically** – no gradient:

$$W_o = TZ^\top (ZZ^\top + \lambda I)^{-1}$$

Step 3 – Subsystem Predicts Each subsystem outputs $Y_s = W_o Z$ independently.

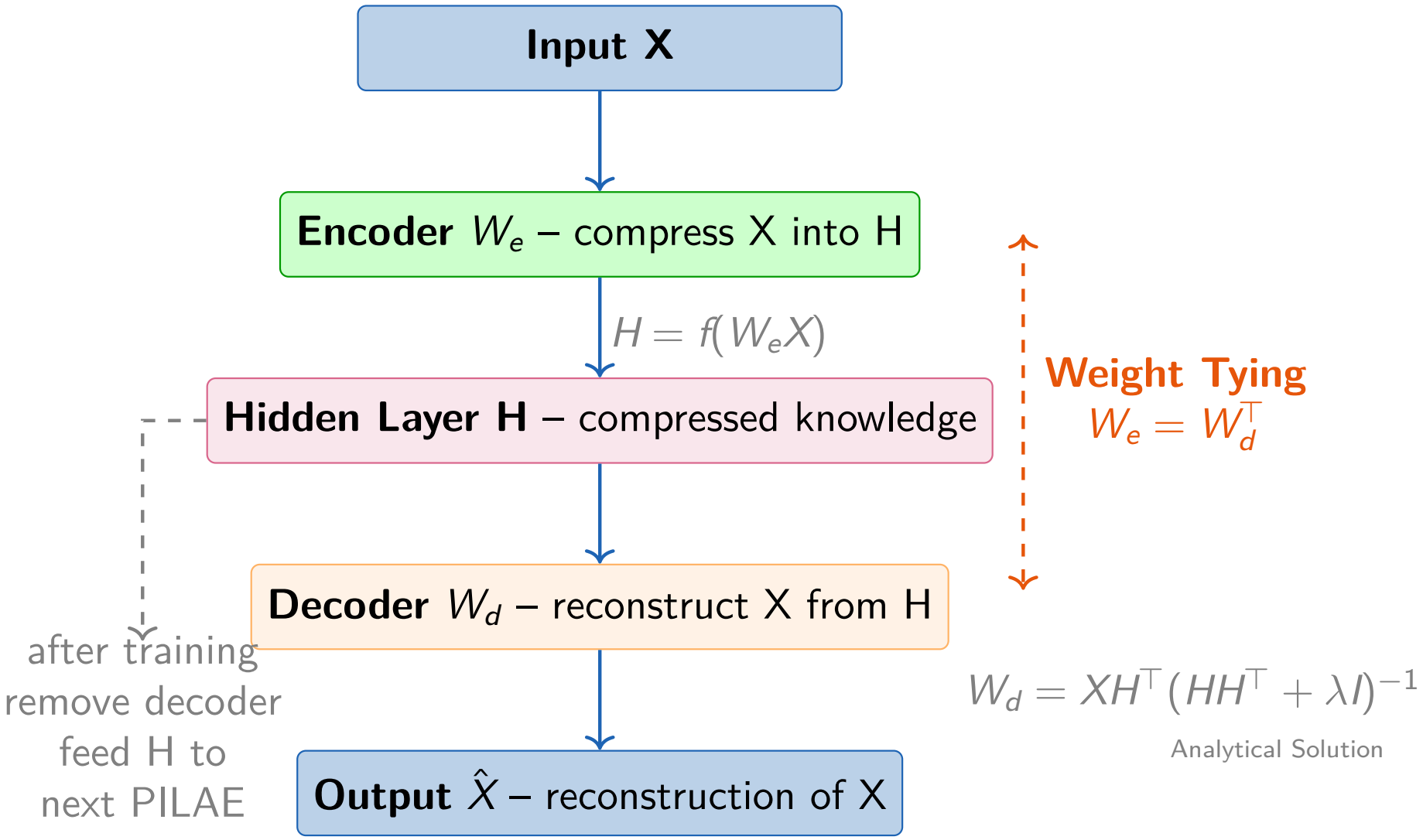
Step 4 – All Vote Together All Y_s values are **added element-wise**.
Highest combined score = final answer.
More subsystems = more stable result.

Forward Learning – How Patterns Are Extracted



accuracy improved? add layer. else stop.

Pseudoinverse Learning Based Autoencoder (PILAE)



Pseudoinverse Learning Based Autoencoder (PILAE)

What is PILAE?

PILAE is one encoder-decoder pair where the decoder weight W_d is solved **analytically in one step** using the Moore-Penrose pseudoinverse formula. No gradient descent needed.

Encoder

Compresses input X into a smaller hidden representation H. Captures the most important patterns in the data.

$$H = f(W_e X)$$

Decoder

Reconstructs X back from H.

$$W_d = XH^\top (HH^\top + \lambda I)^{-1}$$

Weight Tying

Encoder and decoder share the same weights by symmetry. $W_e = W_d^\top$ once W_d is computed, W_e is free. This halves the number of parameters to learn.

Stacking PILAEs

After training one PILAE, the decoder is removed. The hidden output H becomes the input to the next PILAE. Each layer builds a more abstract representation than the last. This is how the forward learning stack is built layer by layer.

From Loss Function to Pseudoinverse Solution

Standard Autoencoder Loss

$$L(W_e, W_d) = \frac{1}{2N} \sum_{i=1}^{N} \|g(W_d f(W_e x_i)) - x_i\|_2^2$$

$\downarrow$ Let $H = f(W_e X)$ + assume decoder is linear + add regularisation λ

PILAE Regularised Loss

$$L(W_d) = \frac{1}{2}\|W_d H - X\|_2^2 + \frac{\lambda}{2}\|W_d\|_r$$

$\downarrow$ Set $r = 2$ + take $\frac{\partial L}{\partial W_d} = 0$ + solve for W_d

Analytical Solution (used in paper)

$$W_d = XH^\top (HH^\top + \lambda I)^{-1}$$

$\downarrow$ By autoencoder symmetry (weight tying)

Encoder Weight – free after decoder is solved

$$W_e = W_d^\top$$

FISTA – Fast Iterative Shrinkage-Thresholding Algorithm

Start: $V_1 = W_d^0,\ t_1 = 1$

Step 1 – Update W_d

$$W_d^k = p_L(V^k)$$

Apply proximal operator to V^k

Step 2 – Update momentum t

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$$

Momentum grows each iteration

Step 3 – Update V

$$V^{k+1} = W_d^k + \frac{t_k - 1}{t_{k+1}}(W_d^k - W_d^{k-1})$$

Look ahead – overshoot slightly

Converged? – check residual

no – repeat

yes

Optimal W_d found

FISTA – Fast Iterative Shrinkage-Thresholding Algorithm

Why FISTA?

When L1 regularisation (LASSO) is used in Eq 2, the loss has a sharp corner at zero – we cannot take a normal derivative. FISTA solves this efficiently.

What is the Proximal Operator p_L?

It handles the non-smooth L1 part by applying soft-thresholding – small weights are pushed to exactly zero, large weights are kept but shrunk slightly.

What is the Momentum t?

FISTA adds a momentum term that grows each iteration. Instead of taking one step toward the answer, it looks ahead and overshoots slightly – reaching the solution much faster than regular ISTA.

FISTA vs ISTA

ISTA takes one small step per iteration. FISTA uses the momentum t to accelerate – converges in far fewer iterations. Like a ball rolling downhill vs a person walking step by step.

In our project

FISTA is used inside each PILAE layer when sparse features are needed. It finds the optimal W_d with L1 regularisation efficiently.

Proximal Operator

Current estimate V^k

↓

Gradient Step

$$V^k - \frac{1}{L}\nabla q(V^k)$$

$q(V) = \|VH - X\|_2^2$
smooth part of loss

Soft Thresholding

$$p_L(V^k) = \text{argmin}_{W_d}\left\{\|W_d\|_1 + \frac{L}{2}\|W_d - \cdot\|_2^2\right\}$$

handles the non-smooth L1 part

Result – Soft Threshold applied

if $|w| > \frac{\lambda}{L}$ – shrink toward zero

if $|w| \leq \frac{\lambda}{L}$ – set to exactly zero

↓

Updated W_d^k – sparse weights

Many weights become exactly zero – automatic feature selection

Proximal Operator

What is the Proximal Operator?

The L1 norm $\|W_d\|_1$ has a sharp corner at zero – we cannot take its derivative normally. The proximal operator handles this by applying soft-thresholding to each weight individually.

The Formula

$$p_L(V) = \underset{W_d}{\operatorname{argmin}} \left\{ \|W_d\|_1 + \frac{L}{2} \left\| W_d - \left(V - \frac{1}{L} \nabla q(V) \right) \right\|_2^2 \right\}$$

where $q(V) = \|VH - X\|_2^2$ is the smooth part and $L = 2E_{max}(HH^\top)$ is the Lipschitz constant.

Soft Thresholding

For each individual weight value w:

$$\text{if } |w| > \frac{\lambda}{L} \quad \rightarrow \text{shrink it toward zero by } \frac{\lambda}{L}$$

$$\text{if } |w| \leq \frac{\lambda}{L} \quad \rightarrow \text{set it to exactly } \textbf{zero}$$

Why this matters

After the proximal step many weights become exactly zero. The model automatically selects only the most important features and ignores the rest. This is called **sparsity** – and it is what makes L1 regularisation powerful.

Alternating Direction Method of Multipliers (ADMM)

Start: initialise W_d, Z, U to zero

Step 1 – Update W_d (smooth part)

$$W_d^{k+1} = (XH^\top + \rho(Z^k - U^k))(HH^\top + \rho I)^{-1}$$

Solve the smooth quadratic part directly

Step 2 – Update Z (sparse part)

$$Z^{k+1} = S_{\lambda/\rho}(W_d^{k+1} + U^k)$$

Apply soft thresholding enforce sparsity

Step 3 – Update U (dual variable)

$$U^{k+1} = U^k + W_d^{k+1} - Z^{k+1}$$

Track the gap between W_d and Z

Converged? $\|W_d - Z\| \approx 0$

no – repeat

yes

Optimal sparse W_d found

Alternating Direction Method of Multipliers (ADMM)

What is ADMM?

ADMM is an alternative solver to FISTA for the same LASSO problem. It splits the problem into two simpler parts and solves them alternately until both agree on the same answer.

The key idea – split W_d into two copies

W_d handles the smooth reconstruction part. Z is a copy of W_d that handles the sparse L1 part. U tracks how far apart W_d and Z are.
Each iteration brings W_d and Z closer together until they are equal – that is the optimal solution.

Soft Thresholding $S_{\lambda/\rho}$

Same idea as the proximal operator in FISTA – weights below the threshold become exactly zero, others are shrunk toward zero.

FISTA vs ADMM in our project

	FISTA	**ADMM**
Speed	Faster convergence	Slightly slower
Sparsity	Good	Slightly sparser
Control	Less	More (ρ parameter)

Our results compare both – FISTA wins on speed, ADMM wins on sparsity.

Backward Learning

Label T

↓

Ideal Last Hidden Layer H_b^l
$$H_b^l = W_o^\dagger T$$

↓

Previous Hidden Layers
$$H_b^{l-1} = (W_e^l)^\dagger \sigma^{-1}(H_b^l)$$

...

First Hidden Layer H_b^1

↓

Backward Weights $W_b^1 \ldots W_b^l$
$$W_b^1 = H_b^1 X^\dagger$$

↓

Backward Prediction Y_b

$\dagger$ = pseudoinverse
start from label
work backwards
repeat back
through all layers
using σ^{-1}

analytically
computed
no gradient

Backward Learning

Why Backward Learning?

Forward learning only uses data reconstruction – it never sees the correct labels during feature extraction. This means it misses features that are specifically useful for classification. Backward learning fixes this by injecting label information directly into the network.

How it works – Step by Step

Once forward learning finishes, the same network structure is reused but in **reverse**.

Step 1 – Start from the label T. Compute the ideal last hidden layer:

$$H_b^l = W_o^\dagger T$$

Step 2 – Propagate backwards through each layer using the inverse activation function σ^{-1}:

$$H_b^{l-1} = (W_e^l)^\dagger \sigma^{-1}(H_b^l)$$

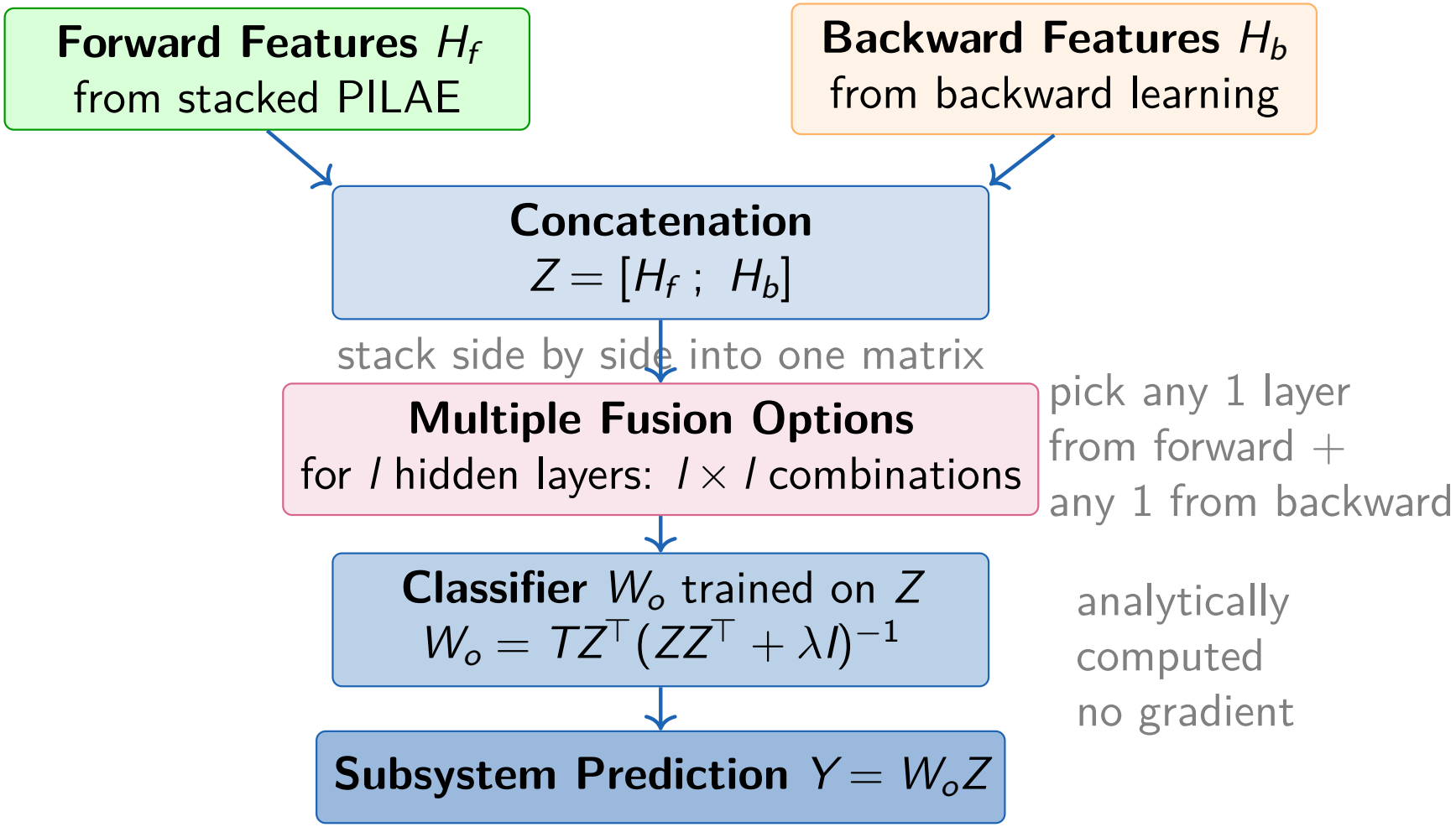
Step 3 – Compute backward weights analytically:

$$W_b^1 = H_b^1 X^\dagger \qquad W_b^2 = H_b^2(\sigma(W_b^1 X))^\dagger$$

Step 4 – Generate backward prediction:

$$Y_b = W_b^{l+1}\sigma(W_b^l\sigma(...\sigma(W_b^1 X)...))$$

Feature Fusion



Feature Fusion

What is Feature Fusion?

Forward and backward learning each produce their own set of features. Feature fusion **concatenates** them side by side into one combined representation Z.

The Fusion Formula

$$Z = [H_f \; ; \; H_b]$$

H_f = forward features – what the data looks like.
H_b = backward features – what makes each class different.
Z = both stacked side by side into one matrix.

Multiple Fusion Combinations

For a network with l hidden layers there are $l \times l$ possible combinations – pick any one layer from the forward path and any one layer from the backward path. Alternatively multiple layers from both paths can be fused together for even richer features.

Classifier on Fused Features

The final classifier weight is computed analytically on the fused features Z:

$$W_o = TZ^\top (ZZ^\top + \lambda I)^{-1}$$

Toy Example – Step 1: Input Data

The Dataset

Sample	Feature 1	Feature 2	Class
x_1	1	0	Class 1
x_2	0	1	Class 2

Input Matrix X and Label Matrix T

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

What is one-hot encoding?

Instead of saying class $= 1$ or class $= 2$, we write:
Class $1 = [1,\ 0]$ Class $2 = [0,\ 1]$
Each column of T is the label for one sample.

Toy Example – Steps 2 & 3: Forward Learning

Step 2 – Random Encoder Weight W_e

We randomly initialise the encoder weight:

$$W_e = \begin{bmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{bmatrix}$$

Step 3a – Compute Hidden Layer H

Apply encoder to input, then pass through tanh activation:

$$W_e X = \begin{bmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{bmatrix} \qquad H = \tanh(W_e X) \approx \begin{bmatrix} 0.537 & 0.379 \\ 0.291 & 0.604 \end{bmatrix}$$

Step 3b – Compute Decoder Weight W_d (Pseudoinverse)

Using the analytical formula – no gradient, one step:

$$W_d = XH^\top (HH^\top + \lambda I)^{-1} \approx \begin{bmatrix} 1.31 & -0.82 \\ -0.63 & 1.48 \end{bmatrix}$$

Step 3c – Weight Tying

By autoencoder symmetry, the encoder weight is updated:

$$W_e^{\text{new}} = W_d^\top = \begin{bmatrix} 1.31 & -0.63 \\ -0.82 & 1.48 \end{bmatrix}$$

Toy Example – Steps 4 & 5: Forward Features & Output Weight

Step 4 – Compute Forward Encoded Features H_f

Using the updated encoder weight W_e^{new}:

$$H_f = \tanh(W_e^{\text{new}} X) \approx \begin{bmatrix} 0.86 & -0.55 \\ -0.67 & 0.90 \end{bmatrix}$$

What this means

The two input samples have been transformed into a new 2D space:
$x_1 \mapsto [0.86,\ -0.67] \quad x_2 \mapsto [-0.55,\ 0.90]$
The samples are now more **separated** – easier to classify.

Step 5 – Compute Output Weight W_o

The classifier weight is solved analytically on the forward features:

$$W_o = TH_f^\top (H_f H_f^\top + \lambda I)^{-1}$$

Key point

Every single weight computed so far – W_d, W_e, W_o – was calculated using **one formula each**. No gradient. No iterations. No learning rate.

Toy Example – Steps 6 & 7: Backward Learning & Fusion

Step 6a – Backward Start: Ideal Hidden State

Push the label T backwards through W_o using pseudoinverse:

$$H_b^l = W_o^\dagger T \quad \text{then} \quad H_b^{l\,\prime} = \operatorname{atanh}(H_b^l) \quad \text{(inverse activation)}$$

Step 6b – Backward Encoded Features H_b

Apply encoder pseudoinverse to get backward features:

$$H_b = (W_e^{\text{new}})^\dagger H_b^{l\,\prime} \approx \begin{bmatrix} 0.6 & -0.6 \\ -0.6 & 0.6 \end{bmatrix}$$

Step 7 – Feature Fusion

Stack forward and backward features side by side:

$$F = [H_f\,;\ H_b] = \begin{bmatrix} 0.86 & -0.55 \\ -0.67 & 0.90 \\ 0.6 & -0.6 \\ -0.6 & 0.6 \end{bmatrix}$$

Each sample now has a **4-dimensional** feature vector:
First 2 values describe **what the data looks like** (forward).
Last 2 values describe **what makes it its class** (backward).

Toy Example – Steps 8 & 9: Final Classifier & Prediction

Step 8 – Train Final Classifier on Fused Features

$$W_o^{\text{fused}} = TF^\top (FF^\top + \lambda I)^{-1} \approx \begin{bmatrix} 0.73 & 0.28 \\ 0.99 & 1.51 \\ -0.09 & -0.28 \\ -0.63 & 0.28 \end{bmatrix}$$

Step 9 – Final Prediction

$$Y = W_o^{\text{fused}} \cdot F = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Reading the result

x_1 gives $[1,\ 0] \rightarrow$ **Predicted: Class 1** ✓
x_2 gives $[0,\ 1] \rightarrow$ **Predicted: Class 2** ✓
Both samples classified **correctly on the first try – zero gradient descent**.

Input $\rightarrow$ Random $W_e \rightarrow H$ analytically $\rightarrow W_d$ analytically $\rightarrow H_f \rightarrow$ Backward pass $\rightarrow$ Fuse $[H_f; H_b] \rightarrow W_o^{\text{fused}}$ analytically $\rightarrow$ Predict.
No loops. No learning rate. No gradient.

Datasets

19 benchmark datasets used to test our system

Like a medicine tested on many patients before claiming it works – we test on 19 public datasets across different domains.

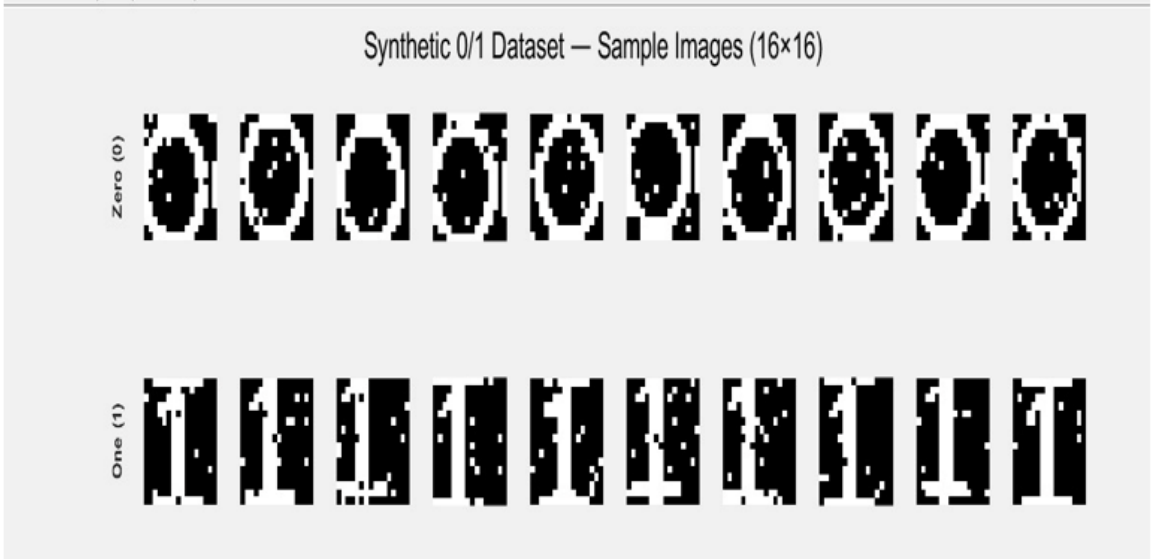
Dataset	Type	Classes	Train	Test	Size
MNIST	Grayscale handwritten digits	10	60,000	10,000	28×28 px
F-MNIST	Grayscale clothing images	10	60,000	10,000	28×28 px
NORB	Stereo 3D object images	5	24,300	24,300	96×96 px

Synthetic 0/1 Dataset

16×16 binary pixel images of digits 0 and 1 with added noise. Used to visually verify the system works before testing on real data.

Synthetic 0/1 Dataset

Property	Value
Image size	16 × 16 pixels
Feature dimension	256
Classes	2 — digit 0 and digit 1
Samples per class	500
Total samples	1,000
Pixel values	Binary — 0 or 1



Synthetic 0/1 Dataset

Property	Value
Image size	16×16 pixels
Feature dimension	256
Classes	2 (digit 0 and 1)
Samples per class	500
Total samples	1,000
Pixel values	Binary (0 or 1)

Added noise

Random pixel flips, blur, and spatial shifts were added to make the task realistic.

What we found

Our model correctly classifies 0s and 1s even with noise – confirming the two-way pseudoinverse approach works before scaling to MNIST and NORB.

Why this validates our system

Known ground truth – errors are obvious.

Controllable difficulty – we can add or remove noise to test robustness.

Visually inspectable – we can see every sample.

Fast to train – confirms the architecture is implemented correctly before heavier datasets.

Result

100% accuracy achieved on the synthetic dataset – confirming that PILAE, two-way learning, and synergetic design all function correctly.

Results – NORB Dataset

Task

Classify stereo grayscale images of 5 object categories: Animal, Human, Airplane, Truck, Car. 96×96 pixels. 48,600 total images. Most challenging of the three image datasets.

Our Configuration

10 subsystems, sampling ratio 0.8
Forward structure: 1500 – 1000 – 600 – 500
Classifier neurons: 5,000
Early stopping applied

Accuracy Comparison

Method	NORB Accuracy
Ours	**91.50%**
PILLS	90.46%
ELM-AE	89.14%
HELM	88.70%
PILAE	88.11%
BLS	86.79%

Key result

Results – MNIST Dataset

Task

Classify 28×28 pixel handwritten digit images (0–9). 70,000 total images.

Our Configuration

2 subsystems, sampling ratio 0.8
Forward structure: 2000 – 1500 – 500
Classifier neurons: 10,000
Early stopping applied

Accuracy Comparison

Method	MNIST Accuracy
Ours	**98.97%**
HELM	98.78%
ELM-AE	98.78%
BLS	98.60%
PILLS	97.48%
PILAE	96.93%

Ensemble Accuracy vs Number of Subsystems

As more subsystems are added, accuracy rises then stabilises. Best result at **2 subsystems: 95.08%** on the ensemble test.

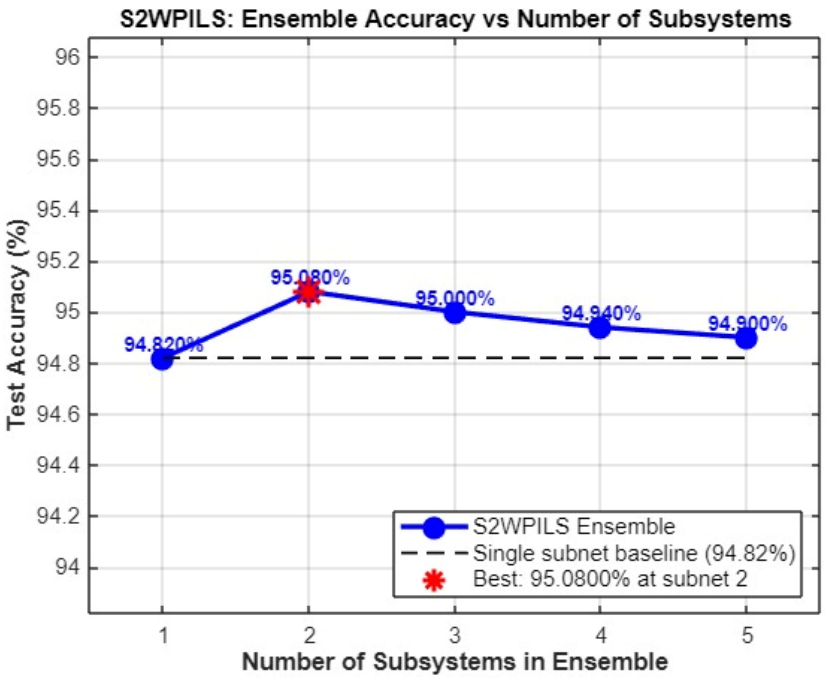
FISTA vs ADMM on MNIST

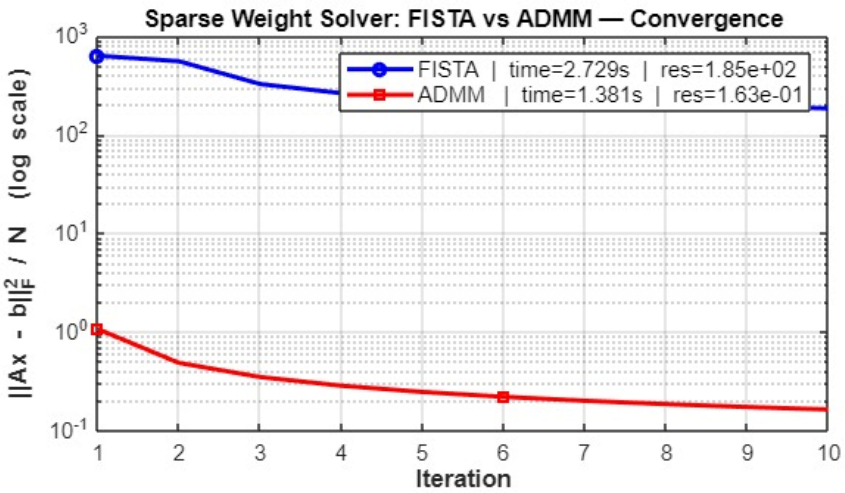
	FISTA	ADMM
Training Time	2.729s	1.381s
Residual	1.85e+02	1.63e-01
Weight Sparsity	0.0%	0.0%

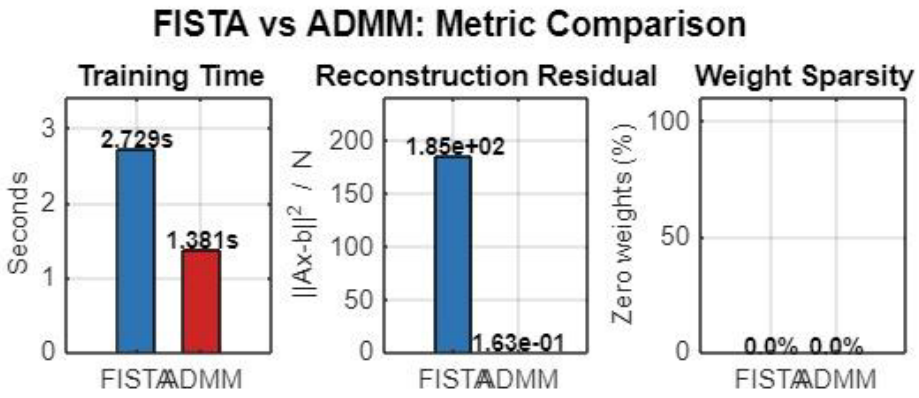
Interpretation

ADMM converges faster for MNIST. FISTA has a higher residual – meaning it leaves more reconstruction error but still achieves top accuracy. Both produce zero sparse weights on this dense dataset.

Results – MNIST Dataset







Results – F-MNIST Dataset

Task

Classify 28×28 greyscale images of 10 clothing categories – shirts, shoes, bags etc. Much harder than MNIST because clothing items look more similar to each other.

Our Configuration

3 subsystems, sampling ratio 0.8
Forward structure: 1500 – 1000 – 600 – 500
Classifier neurons: 10,000
Early stopping applied

Accuracy Comparison

Method	F-MNIST Accuracy
BLS	89.60%
Ours	**89.58%**
ELM-AE	88.60%
HELM	88.53%
PILLS	87.29%
PILAE	86.32%

Key result

We trail BLS by only **0.02%** – effectively tied for first place – but our method trains in a fraction of the time BLS requires.

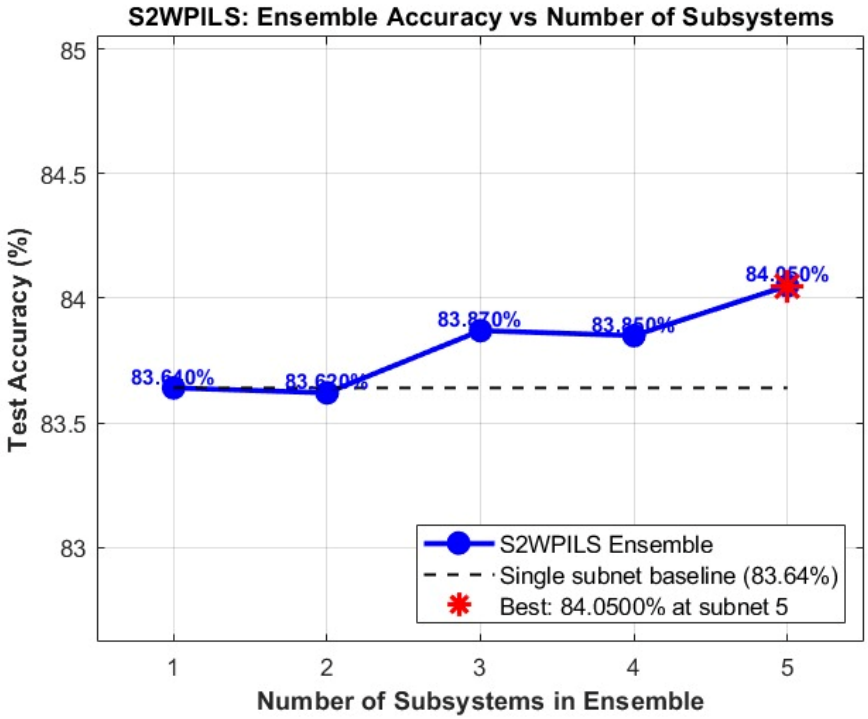
FISTA vs ADMM on F-MNIST

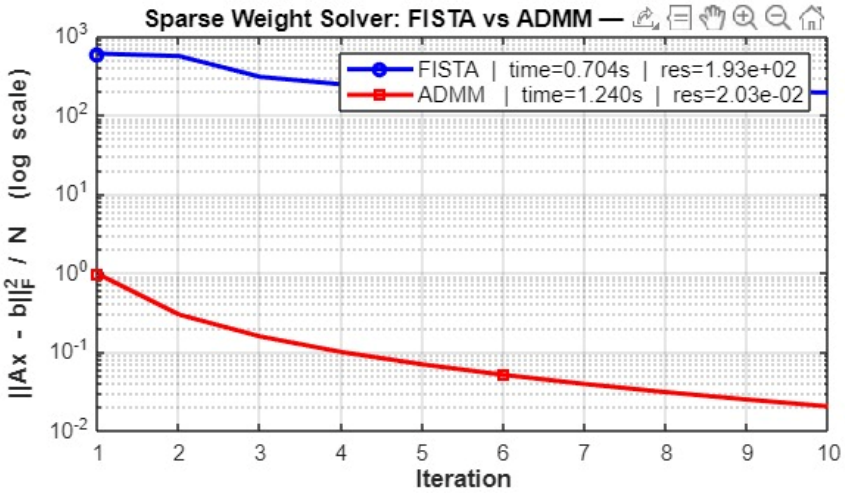
	FISTA	ADMM
Training Time	0.704s	1.240s
Residual	1.93e+02	2.03e-02
Weight Sparsity	0.0%	0.0%

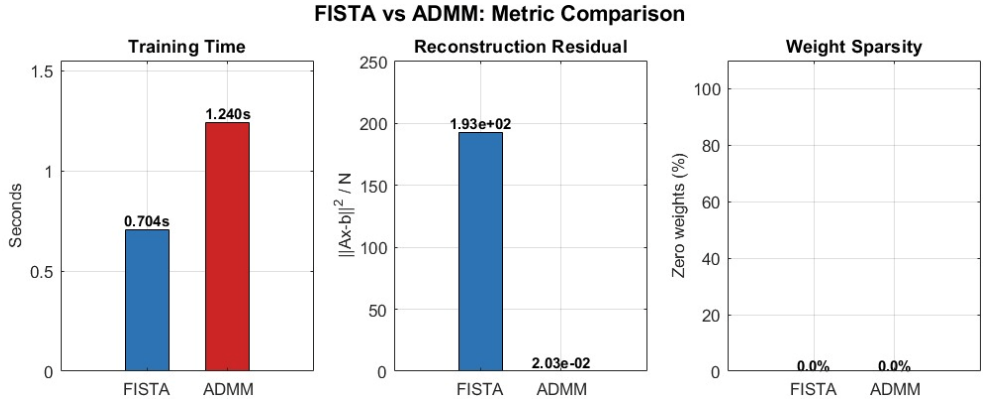
Ensemble Accuracy vs Subsystems

Accuracy grows steadily from 1 to 5 subsystems. Best result at **5 subsystems: 84.05%**. Each additional subsystem contributes complementary features learned from a different random sample of the data.

Results –F-MNIST Dataset







Results – Synthetic 0/1 Dataset

Task

Classify our own 16×16 binary pixel images as digit 0 or digit 1 – with noise.

Ensemble Accuracy vs Subsystems

Even a **single subsystem achieves 100%** accuracy on this simple binary task. Adding more subsystems keeps it at 100% – confirming the architecture is correct.

FISTA vs ADMM on Synthetic 0/1

	FISTA	ADMM
Training Time	0.367s	0.654s
Residual	5.09e+01	4.45e-01
Weight Sparsity	0.0%	0.0%

What this validates

The pseudoinverse approach, two-way learning, and synergetic design all work correctly even on a custom self-made dataset – not just on famous benchmarks.

FISTA wins on speed

FISTA converges in 0.367s vs ADMM's 0.654s. ADMM achieves a much lower residual – reconstructing the data more accurately.

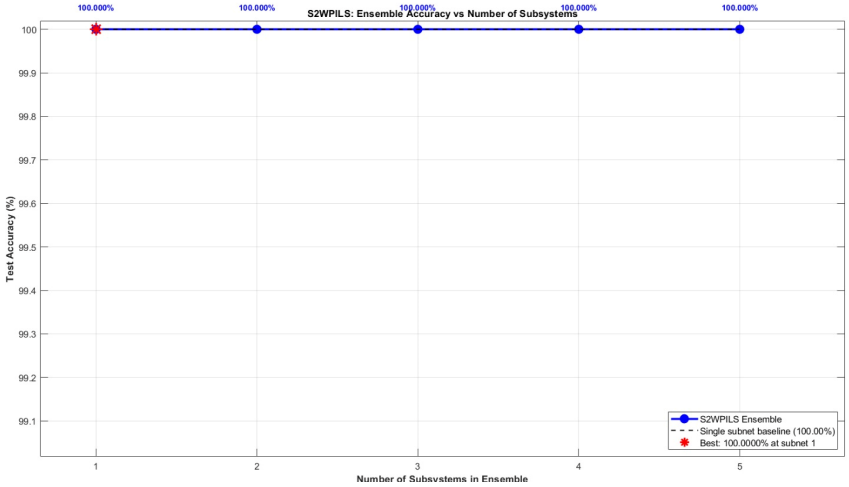
Convergence behaviour

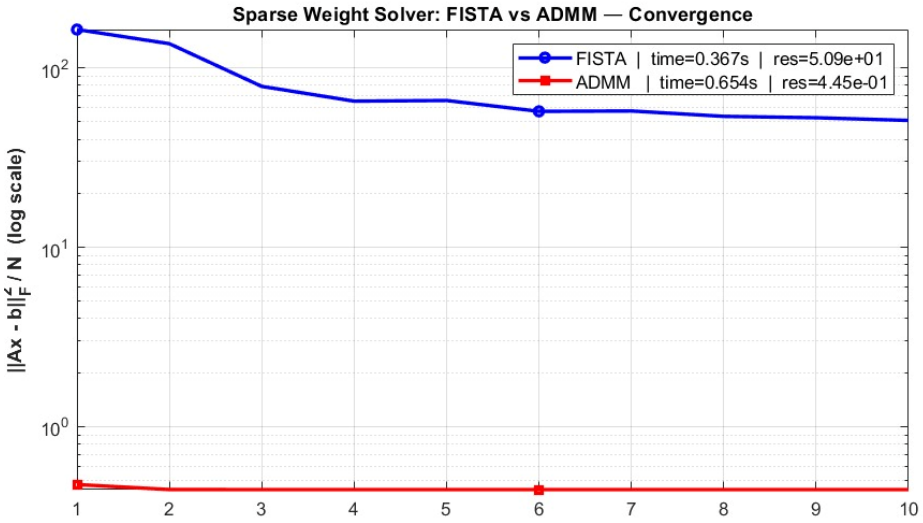
The curves on this simple dataset are much steeper than on MNIST – confirming that harder data requires more iterations. The system **scales gracefully** with task difficulty.

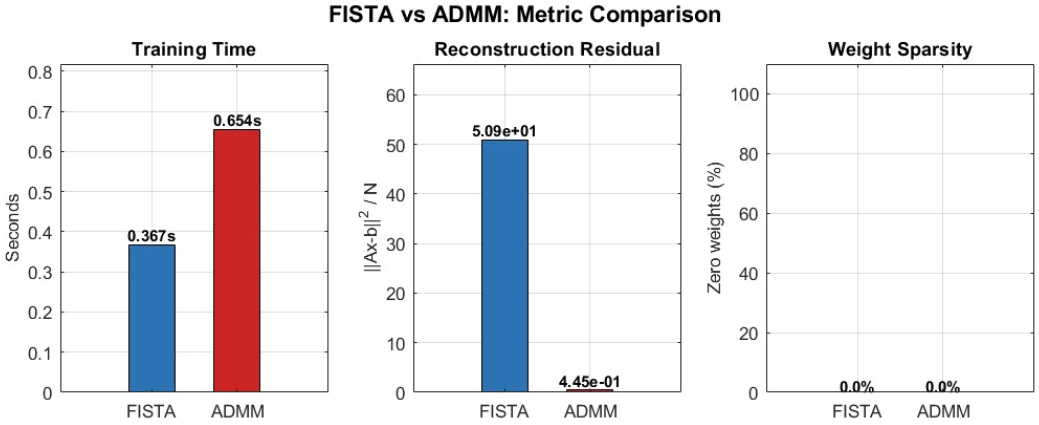
Bottom line

100% accuracy. Both solvers work. Our implementation is verified end-to-end.

Results – Synthetic 0/1 Dataset







Conclusion

Two-way learning = richer features. Forward + backward learning together extract more useful patterns than either alone. Outperformed all baselines on 16 out of 19 datasets.

Synergetic subsystems = more robust predictions. Multiple subsystems collaborating produce more stable and accurate results than any single model.

No gradient descent = dramatically faster training. Training completes in seconds to minutes vs hours for ResNet50, VGG16, LeNet-5 – at similar or better accuracy.

Adaptive depth = no manual architecture design. Early stopping automatically determines how many layers are needed.

Our contributions

Added **GPU acceleration** using `gpuArray` with CPU–GPU compatibility fixes (`mapminmax`, `orth`, etc.). Created the **synthetic 16×16 binary digit dataset** with noise and spatial variation for testing. Visualised results using **accuracy vs subnets** and **FISTA vs ADMM** comparison plots.